

Module 06: Unsupervised Learning & Pattern Discovery

Lecturer: Abdan Hafidz

Topic: Finding Structure in Chaos

1 Introduction: Learning Without a Teacher

In our previous explorations of Supervised Learning, the computer acted like a student taking a test with an answer key. We provided inputs (features) and the correct outputs (labels), and the machine learned to map one to the other. However, the real world rarely provides an answer key. Data is often messy, chaotic, and completely unlabeled.

To understand **Unsupervised Learning**, consider how a baby learns. If you give a toddler a pile of wooden blocks—some red cubes, some blue spheres, and some yellow pyramids—you do not need to teach them the definitions of "cube" or "sphere." Instinctively, the child will begin to sort them. They might group all the sharp, pointy objects together and all the smooth, round objects in another pile. Or perhaps they will group them by color. Without any external instruction or labels, the child discovers the *latent structure* of the objects. This is exactly what Unsupervised Learning algorithms do: they ingest raw, unlabeled data and mathematically discover the hidden patterns, groups, or "clusters" that define it.

2 K-Means Clustering: The Geometry of Grouping

K-Means is the most fundamental algorithm in the clustering toolkit. Its philosophy is simple: data points that are spatially close to each other likely belong to the same group. The "Mean" in K-Means refers to the centroid, or the geometric center, of a cluster.

The algorithm operates as an iterative refinement process. We begin by deciding how many groups we want to find (K). The algorithm then randomly places K points in the data space to act as temporary centers. Every single data point in the dataset then asks, "Which center am I closest to?" and assigns itself to that group. Once the groups are formed, the centers realize they are not actually in the center of their new groups, so they move to the mathematical average of their members. This process—assign, move, assign, move—repeats until the centers stop moving, a state we call **convergence**.

2.1 Determining the Optimal K

The greatest weakness of K-Means is that it is blind to the "true" number of clusters. If you ask it to find 2 clusters in a dataset that clearly has 5, it will dutifully force the data into 2 groups, resulting in a nonsensical partition. We use two primary mathematical methods to determine the optimal K .

2.1.1 The Elbow Method (Inertia)

The objective of clustering is to minimize the distance between data points and their cluster center. This metric is called **Inertia** or **Sum of Squared Errors (SSE)**.

$$SSE = \sum_{k=1}^K \sum_{x \in C_k} ||x - \mu_k||^2$$

As we increase K , the SSE will naturally decrease. In the extreme case where K equals the number of data points, SSE becomes zero because every point is its own cluster. This is useless. Instead, we look for the "Elbow Point": the value of K where the SSE starts decreasing much more slowly. This represents the best trade-off between compression and accuracy.

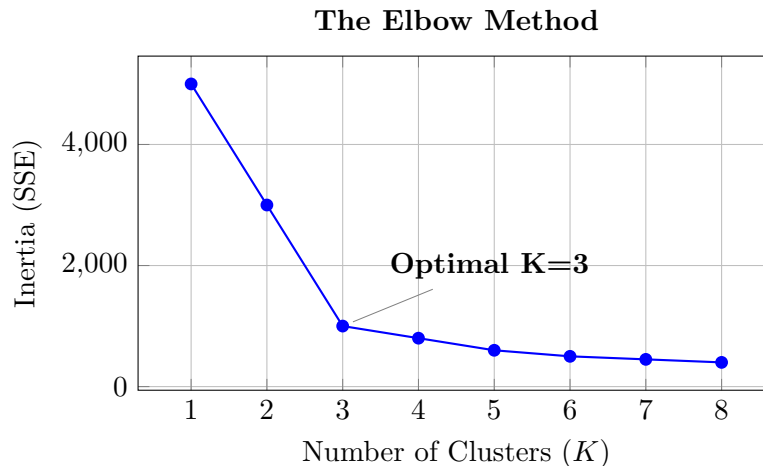


Figure 1: The Elbow Method. Notice the sharp "bend" at $K=3$. Before $K=3$, adding clusters drastically reduces error. After $K=3$, the improvement is marginal.

2.1.2 The Silhouette Score

While Inertia measures how tight a cluster is, it fails to measure how distinct the clusters are from each other. The **Silhouette Score** (S) solves this by comparing *Cohesion* (how close a point is to its own group) against *Separation* (how far it is from the next nearest group).

For a single data point i :

- Let $a(i)$ be the average distance from point i to all other points in its **own** cluster. We want this to be small (high cohesion).
- Let $b(i)$ be the average distance from point i to all points in the **nearest neighboring**

cluster. We want this to be large (high separation).

The score is defined as:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

This formula produces a value between -1 and 1, providing a rigorous interpretation:

- **Approaching +1:** Perfect clustering. Here, $b(i) \gg a(i)$, meaning the point is deeply inside its own cluster and far from neighbors.
- **Approaching 0:** Borderline case. Here, $b(i) \approx a(i)$, meaning the point sits right on the boundary between two clusters. The model is unsure where it belongs.
- **Approaching -1:** Incorrect clustering. Here, $a(i) > b(i)$, meaning the point is actually closer to the neighboring cluster than its own.

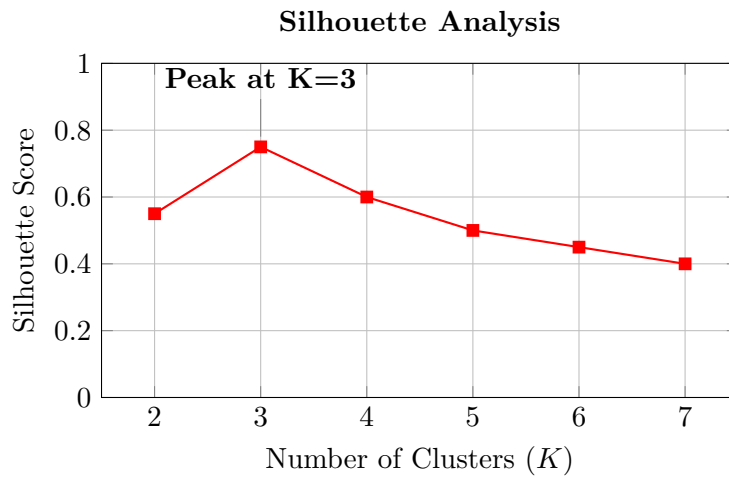


Figure 2: Silhouette Score vs K. Unlike the Elbow method where we look for a bend, here we simply look for the global maximum. K=3 yields the highest score, confirming it as the best choice.

3 Hierarchical Clustering: The Taxonomy of Data

K-Means is iterative, but Hierarchical Clustering is structural. It builds a hierarchy of clusters, often visualized as a tree or "Dendrogram." There are two approaches: Top-Down (Divisive) and the more common **Bottom-Up (Agglomerative)**.

In Agglomerative clustering, we treat every single data point as its own tiny cluster. Then, we find the two closest points and merge them into a pair. We then find the next closest pair (or point) and merge them. We repeat this process, building larger and larger groups, until the entire dataset is merged into one giant cluster.

3.1 Linkage: The Rules of Connection

The crucial decision in this process is how we measure distance between two groups. This is called the **Linkage Criterion**:

1. **Single Linkage:** We measure the distance between the *closest* members of two clusters. This is akin to a "chain reaction" and often produces long, snake-like clusters. It is sensitive to outliers.
2. **Complete Linkage:** We measure the distance between the *farthest* members of two clusters. This forces the algorithm to find compact, spherical clusters and is generally more robust.
3. **Average Linkage:** We calculate the average distance between all pairs of points in the two clusters. This is a balanced middle ground.

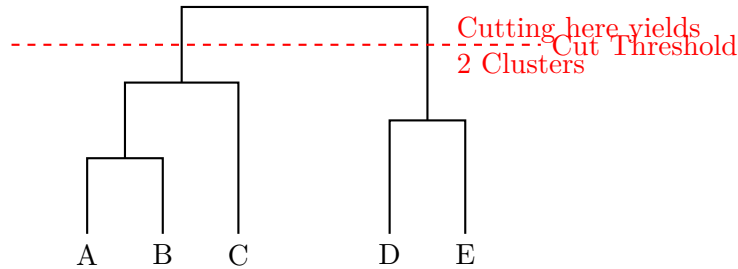


Figure 3: A Dendrogram. The vertical height represents the distance at which clusters are merged. By "cutting" the tree at a specific height (red line), we determine the number of clusters.

4 DBSCAN: Density-Based Clustering

Both K-Means and Hierarchical Clustering struggle with one major issue: they assume data points belong to some cluster. But what about noise? Additionally, K-Means assumes clusters are spherical blobs. In reality, clusters can be crescents, rings, or arbitrary shapes.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) solves this by fundamentally changing the definition of a cluster. Instead of distance to a center, it looks for **Density**. It groups points that are packed closely together and marks points that lie in low-density regions as **Noise (Outliers)**.

It relies on two parameters:

- **Epsilon (ϵ):** The search radius around a point.
- **MinPts:** The minimum number of points required within that radius to form a dense region.

If a point has enough neighbors within ϵ , it becomes a "Core Point" and starts a cluster. If a point is reachable but doesn't have enough neighbors itself, it is a "Border Point." If a point has no neighbors, it is flagged as Noise. This makes DBSCAN exceptionally powerful for anomaly detection.

5 Study Case: Stratified Sampling with Clustering

Let's tie this back to a real-world scenario. Imagine you are working with a massive dataset of 1,000,000 credit card transactions. Crucially, **this dataset is unlabeled**. You don't know which transactions are fraud and which are legit.

Labeling 1,000,000 rows by hand is impossible. Randomly sampling 1,000 rows to label is dangerous because fraud is rare (maybe 1%). A random sample might miss the fraud entirely, leaving you with a training set of only normal data.

The Solution: Unsupervised Stratified Sampling. 1. **Cluster First:** Run K-Means (or MiniBatchKMeans) on the entire unlabeled dataset with a high K (e.g., $K = 50$). The algorithm will group mathematically similar transactions together. 2. **Assumption:** Transactions in the same cluster likely share the same label (e.g., Cluster 42 might contain all the "high-value foreign transactions"). 3. **Sample Representative Points:** Instead of sampling randomly from the whole pool, you select the 50 points closest to the centroid of each cluster. 4. **Label Efficiently:** You now only label these representative points. Since each cluster captures a different "type" of behavior, your small labeled dataset effectively covers the entire diversity of the original data. This guarantees that even rare anomalies (which would form their own small clusters) are captured in your training set.

6 Implementation Guide

```
1 from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
2 from sklearn.metrics import silhouette_score
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 # 1. K-Means Clustering
7 # -----
8 # We iterate through K=2 to K=10 to find the Elbow and Silhouette peaks
9 inertias = []
10 silhouettes = []
11 K_range = range(2, 11)
12
13 for k in K_range:
14     kmeans = KMeans(n_clusters=k, random_state=42)
15     labels = kmeans.fit_predict(X)
16
17     inertias.append(kmeans.inertia_) # For Elbow Method
18     silhouettes.append(silhouette_score(X, labels)) # For Silhouette Score
19
20 # 2. DBSCAN
21 # -----
22 # No K required. We define density instead.
23 # eps=0.5: neighborhood radius
24 # min_samples=5: meaningful cluster size
25 dbscan = DBSCAN(eps=0.5, min_samples=5)
26 labels_db = dbscan.fit_predict(X)
```

```

27
28 # Count outliers (Label is -1)
29 n_noise = list(labels_db).count(-1)
30 print(f"DBSCAN found {n_noise} noise points.")
31
32 # 3. Hierarchical Clustering
33 # -----
34 # Agglomerative approach with Complete Linkage (compact clusters)
35 hc = AgglomerativeClustering(n_clusters=3, linkage='complete')
36 labels_hc = hc.fit_predict(X)

```